Documentatation

Ctxjs

- new-context()

new-context

Creates a new empty context and loads load bytes into it

```
let current-context = ctxjs.new-context(
    ctxjs.load.eval("function preloaded()
{ return true; }")
)
ctxjs.ctx.eval(current-context,
```

```
"preloaded()")
```

```
(<module>, true)
```

Parameters

```
new-context(..load: bytes) -> bytes
```

..load `bytes`

load bytes `created by ctxjs.load.*`

Ctx

- call-function()
- call-module-function()
- define-vars()
- eval()
- eval-format()
- get-module-properties()
- get-module-property()
- load()
- load-module-bytecode()
- load-module-js()

call-function

Calls a js function by function name with an args.

```
let (current-context,_) = ctxjs.ctx.eval(
  current-context,
  "function fnname() { return 1; }",
  transition: true
)
ctxjs.ctx.call-function(
  current-context,
  "fnname",
  varname: "value"
)
```

```
(<module>, 1)
```

Parameters

```
call-function(
  ctx: <module>,
  fnname: str,
  ..args: any,
  transition: bool,
  catch: bool
) -> (<module>, any)
```

ctx `<module>`

the context in which this function should run

fnname `str`

the function name

..args `any`

the args for the function

transition `bool`

if a new context should be created (with changed data)

Default: `false`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

call-module-function

Calls a js function in a module by function name with an args.

```
let (current-context,_) =  ctxjs.ctx.load-module-js(
  current-context,
  "test_module",
  "export function test() {return \"foo\";}",
)
ctxjs.ctx.call-module-function(
  current-context,
  "test_module",
  "test",
  varname: "value"
)
```

```
(<module>, "foo")
```

Parameters

```
call-module-function(
  ctx: <module>,
  modulename: str,
  fnname: str,
  ..args: any,
  transition: bool,
  catch: bool
) -> (<module>, any)
```

ctx `<module>`

the context in which this function should run

modulename `str`

the module name

fnname `str`

the function name

..args `any`

the args for the function

transition `bool`

if a new context should be created (with changed data)

Default: `false`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

define-vars

Defines vars in a new context with given values.

```
ctxjs.ctx.define-vars(
  current-context,
  varname: "value"
)
```

```
(<module>, none)
```

Parameters

```
define-vars(
  ctx: <module>,
  ..vars: any,
  catch: bool
) -> (<module>, none)
```

ctx `<module>`

the context in which this function should run

..vars `any`

the context in which this function should run

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

eval

Evaluates the js code. It will returns a new context if transition is enabled, the current context if not and the value which returns from the js code.

```
ctxjs.ctx.eval(
  current-context,
  "1+1"
)
```

```
(<module>, 2)
```

Parameters

```
eval(
  ctx: <module>,
  js: str bytes,
  transition: bool,
  catch: bool
) -> (<module>, any)
```

ctx `<module>`

the context in which this function should run

js `str` or `bytes`

the js code which should be evaluate

transition `bool`

if a new context should be created (with changed data)

Default: `false`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

eval-format

Evaluates the js code. It will returns a new context if transition is enabled, the current context if not and the value which returns from the js code. Additional to `eval()` it supports formatting of the eval code with typst values.

```
ctxjs.ctx.eval(
  current-context,
  "1+1"
)
```

```
(<module>, 2)
```

Parameters

```
eval-format(
  ctx: <module>,
  js: str bytes,
  ..args: any,
  transition: bool,
  catch: bool
) -> (<module>, any)
```

ctx `<module>`

the context in which this function should run

js `str` or `bytes`

the js code which should be evaluate

..args `any`

named args which replaces the name in the js code with the typst value as js value, only characters a-zA-Z0-9_- as name are allowed

transition `bool`

if a new context should be created (with changed data)

Default: `false`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

get-module-properties

Get all properties from a module.

```
let (current-context,_) =  ctxjs.ctx.load-module-js(
  current-context,
  "test_module",
  "export const key1 = 1;export const key2 = 1;",
)
ctxjs.ctx.get-module-properties(
  current-context,
  "test_module",
)
```

```
(<module>, ("key1", "key2"))
```

Parameters

```
get-module-properties(
  ctx: <module>,
  modulename: str,
  catch: bool
) -> (<module>, array)
```

ctx `<module>`

the context in which this function should run

modulename `str`

the module name

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

get-module-property

Get a property from a module.

```
let (current-context,_) =  ctxjs.ctx.load-module-js(
  current-context,
  "test_module",
  "export const key1 = 1;export const key2 = 1;",
)
ctxjs.ctx.get-module-property(
  current-context,
  "test_module",
  "key1",
)
```

```
(<module>, 1)
```

Parameters

```
get-module-property(
  ctx: <module>,
  modulename: str,
  propertyname: str,
  catch: bool
) -> (<module>, any)
```

ctx `<module>`

the context in which this function should run

modulename `str`

the module name

propertyname `str`

the property name

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

load

Loads the load bytes and always creates a new context.

```
ctxjs.ctx.load(
  current-context,
  ctxjs.load.eval("function fn() {}"),
)
```

```
(<module>, none)
```

Parameters

```
load(
  ctx: <module>,
  ..load: bytes,
  catch: bool
) -> (<module>, none)
```

ctx `<module>`

the context in which this function should run.

..load `bytes`

load bytes `created by ctxjs.load.*`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

load-module-bytecode

Loads bytecode into a new context.

```
ctxjs.ctx.load-module-bytecode(
  current-context,
  read("bytecode.kbc1", encoding: none),
)
```

```
(<module>, none)
```

Parameters

```
load-module-bytecode(
  ctx: <module>,
  bytecode: bytes,
  catch: bool
) -> (<module>, none)
```

ctx `<module>`

the context in which this function should run

bytecode `bytes`

the bytecode mostly created by the `ctxjs_module_bytecode_builder`

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

load-module-js

Loads js module code into a new context.

```
ctxjs.ctx.load-module-js(
  current-context,
  "test_module",
```

```
  "export function test() {}",
)
```

```
(<module>, none)
```

Parameters

```
load-module-js(
  ctx: <module>,
  modulename: str,
  module: str bytes,
  catch: bool
) -> (<module>, none)
```

ctx `<module>`

the context in which this function should run

modulename `str`

the module name

module `str` or `bytes`

the js module code

catch `bool`

if all errors should be catched and return as an dictionary(err:str)

Default: `false`

Load

- call-function()
- call-module-function()
- define-vars()
- eval()
- eval-format()
- load-module-bytecode()
- load-module-js()

call-function

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.call-function()` at loading.

```
ctxjs.load.call-function("fnname", 1)
```

```
bytes(12)
```

Parameters

```
call-function(
  fnname,
  ..args
) -> bytes
```

call-module-function

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.call-module-function()` at loading.

```
ctxjs.load.call-module-
function("example_module", "text", arg1:
1)
```

```
bytes(24)
```

Parameters

```
call-module-function(
  modulename,
  fnname,
  ..args
) -> bytes
```

define-vars

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.define-vars()` at loading.

```
ctxjs.load.define-vars(var1: 1, var2: 2)
```

```
bytes(15)
```

Parameters

```
define-vars(..vars) -> bytes
```

eval

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.eval()` at loading.

```
ctxjs.load.eval("function fn() {}")
```

```
bytes(19)
```

Parameters

```
eval(
  js,
  strict
) -> bytes
```

eval-format

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.eval-format()` at loading.

```
ctxjs.load.eval-format("function() {return
value;}", value: 1)
```

```
bytes(40)
```

Parameters

```
eval-format(
  js,
  ..args,
  strict
) -> bytes
```

load-module-bytecode

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.eval()` at loading.

```
ctxjs.load.load-module-
bytecode(read("bytecode.kbc1"))
```

```
bytes(45)
```

Parameters

```
load-module-bytecode(bytecode) -> bytes
```

load-module-js

Creates load bytes for `ctxjs.new-context()` or `ctx.load()`. Same function as `ctx.load-module-js()` at loading.

```
ctxjs.load.load-module-js(
  "example_module",
  "export function test() {}",
```

```
)
```

```
bytes(46)
```

Parameters

```
load-module-js(
  modulename,
  module
) -> bytes
```

Value

- eval()
- eval-format()
- image-data-url()
- json()
- raw-bytes()

eval

Typst **does not** support a js function as a returning value: `#ctxjs.ctx.eval(ctx, "function() {}")`. So as an alternative `ctxjs.value.eval` can be used, which returns a special formated bytes (`$ctxjs_cbor_` + tagged cbor) with raw js code.

```
#ctxjs.ctx.call-function(ctx, "fnname", (ctxjs.value.eval("function(args) { return true; }"),))
```

translates on the js side to

```
fnname(function(args){ return true; })
```

```
ctxjs.value.eval("1+2")
```

```
bytes(25)
```

Parameters

```
eval(js: str bytes) -> bytes
```

js `str` or `bytes`

the js code which should be evaluate

eval-format

Similar to `eval()` the function returns a special formated bytes (`$ctxjs_cbor_` + tagged cbor) with raw js code. Additional it supports formatting of the eval code with typst values.

```
ctxjs.value.eval-format("{val1}+{val2}", val1: 1, val2: 2)
```

```
bytes(49)
```

Parameters

```
eval-format(
  js: str bytes,
  ..args: any
) -> bytes
```

js `str` or `bytes`

the js code which should be evaluate

..args `any`

named args which replaces the name in the js code with the typst value as js value, only characters a-zA-Z0-9_- as name are allowed

image-data-url

Returns a data url from an image.

```
ctxjs.value.image-data-url(bytes("<svg></svg>"))
```

```

```

Parameters

```
image-data-url(
  data: bytes,
  format: bytes
) -> str
```

data `bytes`

the png, jpg, gif or svg image as bytes

format `bytes`

can be used to override and pre set the image format, if format is `auto` the format with automatically detected

Default: `auto`

json

Similar to `eval()` the function returns a special formated bytes (`$ctxjs_cbor_` + tagged cbor) but with raw json code which will be also validated as pure json code in ctxjs.

```
ctxjs.value.json("{}")
```

```
bytes(24)
```

Parameters

```
json(json: str bytes) -> bytes
```

json `str` or `bytes`

json code

raw-bytes

If the data contains a `$ctxjs_cbor_` header the value gets interpreted as a cbor value. This is currently a workaround, because typst currently doesn't support directly tagged cbor data. To "escape" your bytes and send them as raw bytes with a `$ctxjs_cbor_` header use this function.

```
ctxjs.value.raw-
bytes(bytes("$ctxjs_cbor_should_be_not_inter
```

```
bytes(60)
```

Parameters

```
raw-bytes(b: bytes) -> bytes
```

b `bytes`

the bytes that shoud not interpreted as a cbor value

ctxjs_module_bytecode_builder

Can be simple called like this:

```
cargo run --manifest-path typst-ctxjs-package/Cargo.toml --bin
ctxjs_module_bytecode_builder modulename js/dist/modulecode.js typst-package/
bytecode.kbc1
```

It uses the modulename to compile the module code and write the bytecode to the kbc1 file.

Guide

example files

- example/main.js

```
function create_svg(data) {
    return `<svg width="100" height="100" xmlns="http://www.w3.org/2000/svg"
xmlns:xlink="http://www.w3.org/1999/xlink">
    <image width="100\" height="100" xlink:href="` + data + `"/>
</svg>`
}
```

- example/Typst.svg.png

new context

Creates a new context to work with and preloaded with a simple evaluated js file and a function. It is recommend to build your own js file to an esm file and create bytecode from it via ctxjs_module_bytecode_builder.

```
#import "@preview/ctxjs:0.4.0"
#import ctxjs.load
#import ctxjs.ctx
#import ctxjs.value

#let current-context = ctxjs.new-context(
  load.eval(read("examples/main.js", encoding: none)),
  load.eval("let counter = 0; function changes_data() { counter++; return
counter; }"),
  load.eval("function call_callback(callback) { return "Hello " + callback(); }"),
  load.eval("function pure_json(json) { return json.arr; }"),
  // load.load-module-bytecode(read("main.kbc1", encoding: none)),
)
```

working with context

On calling any ctx function a new or the current context gets returned with a value as an array. Its recommend always use the destructuring syntax to be safe that we are always use the correct context.

```
#let (current-context, value) =
ctx.eval(current-context, "123")
#value
```

123

If you do not need the value, its safe to ignore it:

```
#let (current-context, _) =
ctx.eval(current-context, "123")
value gets ignored
```

```
value gets ignored
```

use a context as a state

Here a small example how to work with a state (just keep in mind that a state key needs to be unique something like this: `@preview/module-name:key`

```
// store an array with a context and a
"none" value
#let global-context = state("@preview/
ctxjs:context", (
  ctxjs.new-context(),
  none,
))
#global-context.update(current-state => {
  let (current-context, _) = current-state
  let (current-context, _) =
ctx.eval(current-context, "let counter_0 =
0", transition: true)
  // writes the (context, value) back to
the state
  return ctx.eval(current-context, "let
counter_1 = 0", transition: true)
})

Value is:
#context {
  let (current-context, _) = global-
context.get()
  let (current-context, value) =
ctx.eval(current-context, "counter_1")
  value
}
```

```
Value is: 0
```

image-data-url

Calls a javascript which takes a base64 image and returns a svg string embedding the base64 image.

```
#let (current-context, value) = ctx.call-
function(current-context, "create_svg",
(value.image-data-url(read("examples/
Typst.svg.png", encoding: none))))

#image(logo_svg)
#str(logo_svg).slice(0,
5)...#str(logo_svg).slice(100,
120)...#str(logo_svg).slice(145, 170)...
```

typst

```
<svg …/xlink">
    <image …xlink:href="data:image/pn…
```

eval-format

Evaluate js directly with formatting data.

```
#let (current-context, value) = ctx.eval-
format(current-context, "`Result for
{val1}+{val2} is ${{val1}+{val2}}`", val1:
1, val2: 2)
#value
```

```
Result for 1+2 is 3
```

transition

A small example to show the difference with transition and without transition.

```
// new context must be stored
#let (current-context, result) = ctx.call-
function(current-context, "changes_data",
transition: true)
// With transition:
#result

//changes_data gets called counter will
increased and returned. The counter state
will be saved with transition.

// Without transition:
#let (current-context, result) = ctx.call-
function(current-context, "changes_data")
#result

//changes_data gets called counter will
increased and returned. Now the counter
will keep the same state as if the
function had never been called.

// its mostly recommend always store the
returning context (just to be safe)
#let (current-context, result) = ctx.call-
function(current-context, "changes_data")

// After:
#result

//Because the counter was not change its
still the same as the last call.
```

```
1

2

2
```

value

Sometimes you want to pass some special data into the javascript. Like a function callback or a pure json document.

This is possible via the value module.

```
#ctx.eval-format(current-context,
"call_callback({test})", test:
value.eval("function() {return
`world`;}"))
```

```
(<module>, "Hello world")
```

```
#ctx.eval-format(current-context,
"pure_json({test})", test:
value.json("{\"arr\":[1,2,3,4]}"))
```

```
(<module>, (1, 2, 3, 4))
```